

CSE 4631

Digital Signal Processing

Week 16

Windowed-Sinc Filters

- They are used to separate one band of frequencies from another
- They are very stable, produce few surprises, and can be pushed to incredible performance levels.
- Performs poorly in the time domain
 - Excessive ripple and overshoot in the step response

Strategy of Windowed-Sinc

- Consider the frequency response of the *ideal* low-pass filter.
- Taking the inverse Fourier transform of this ideal frequency response produces the ideal filter kernel (impulse response).
- This curve is of the general form $\sin(x)/x$, called the **sinc function**, given by:

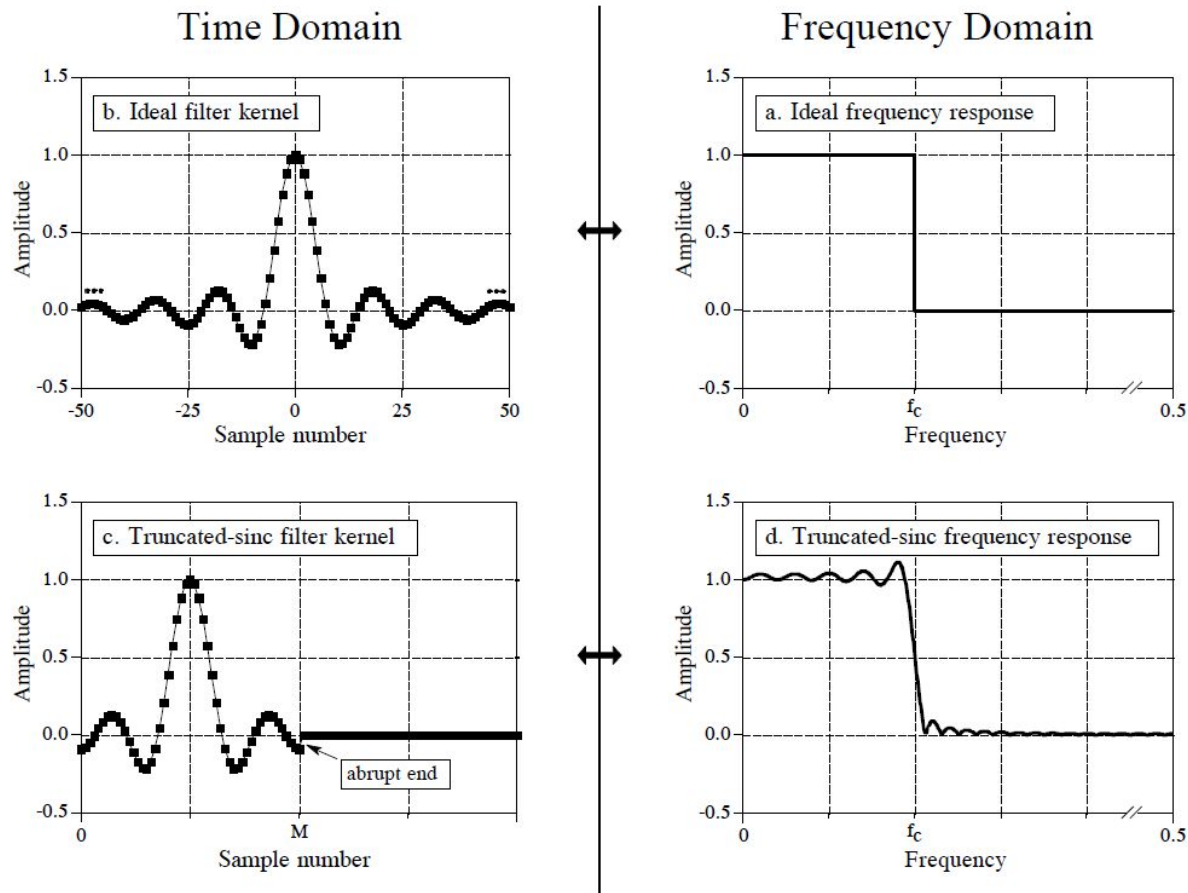
$$h[i] = \frac{\sin(2\pi f_c i)}{i\pi}$$

- Convolution of an input signal with this filter kernel provides a *perfect* low-pass filter.
- Problem is, the sinc function continues to both negative and positive infinity without dropping to zero amplitude.
 - This infinite length is not a problem for mathematics, it is a show stopper for computers.

Strategy of Windowed-Sinc (contd.)

- To get around this problem, we will make **two** modifications to the sinc function.
 - First, it is truncated to $M+1$ points, symmetrically chosen around the main lobe, where M is an even number. All samples outside these $M+1$ points are set to zero, or simply ignored.
 - Second, the entire sequence is shifted to the right so that it runs from 0 to M . This allows the filter kernel to be represented using only *positive* indexes.
- Since the modified filter kernel is only an approximation to the ideal filter
- The frequency response for this modified filter kernel is a mess!
 - Excessive ripple and poor attenuation in the stopband (gibbs effect)
 - This is a result of the abrupt discontinuity at the ends of the truncated sinc function

Strategy of Windowed-Sinc (contd.)



Strategy of Windowed-Sinc (contd.)

- There is a simple method of improving this situation – windows!
 - Blackman window, hamming window, etc.
- Multiplying the truncated-sinc by a window results in the **windowed-sinc** filter kernel.
 - The idea is to reduce the abruptness of the truncated ends and thereby improve the frequency response
 - The passband is now flat (no ripple) and the stopband attenuation is good
- There are **two windows** worth using:
 - The **Hamming window**: $w[i] = 0.54 - 0.46 \cos(2\pi i/M)$
 - The **Blackman window**: $w[i] = 0.42 - 0.5 \cos(2\pi i/M) + 0.08 \cos(4\pi i/M)$

Strategy of Windowed-Sinc (contd.)

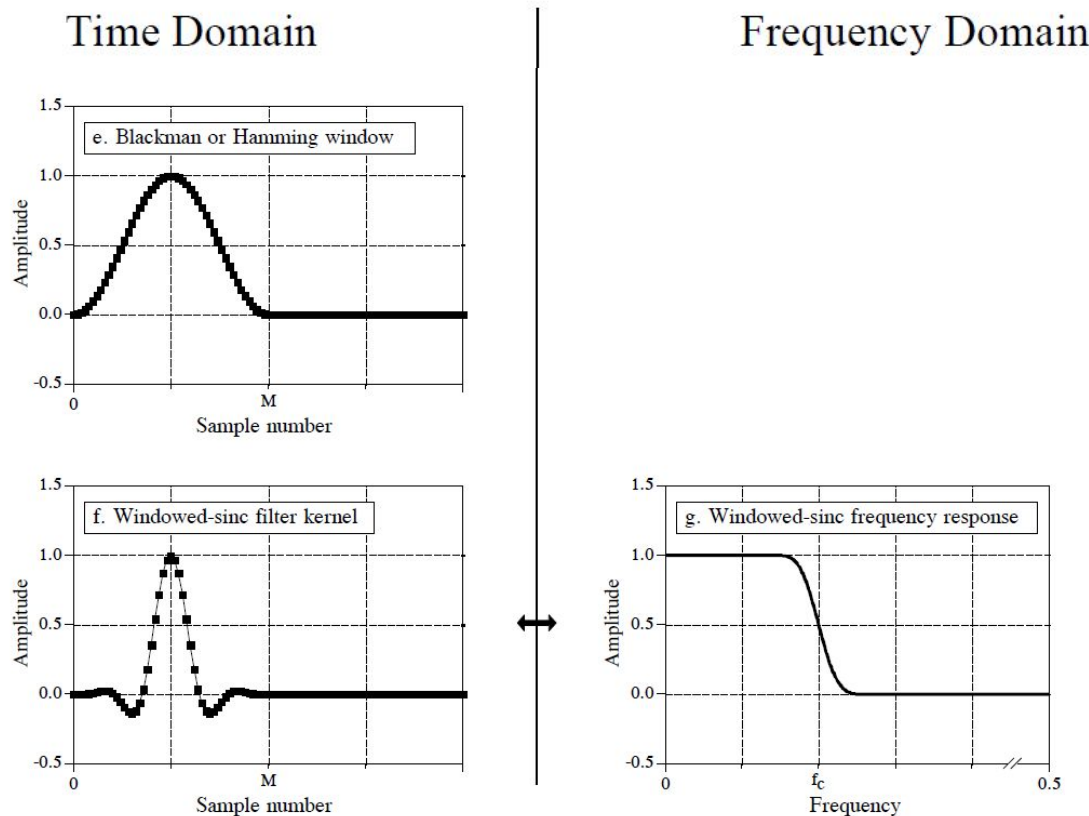


FIGURE 16-1

Strategy of Windowed-Sinc (contd.)

Which of these two windows should you use?

It's a trade-off between parameters:

- The Hamming window has about a 20% *faster roll-off* than the Blackman.
- The Blackman has a *better stopband attenuation* and a *lower passband ripple*.
 - The stopband attenuation for the Blackman is $\sim 74\text{dB}$ (0.02%), while the Hamming is only $\sim 53\text{dB}$ (-0.2%).
 - The Blackman has a passband ripple of only about 0.02%, while the Hamming is typically 0.2%.

In general, the **Blackman should be your first choice**; a slow roll-off is easier to handle than poor stopband attenuation.

Strategy of Windowed-Sinc (contd.)

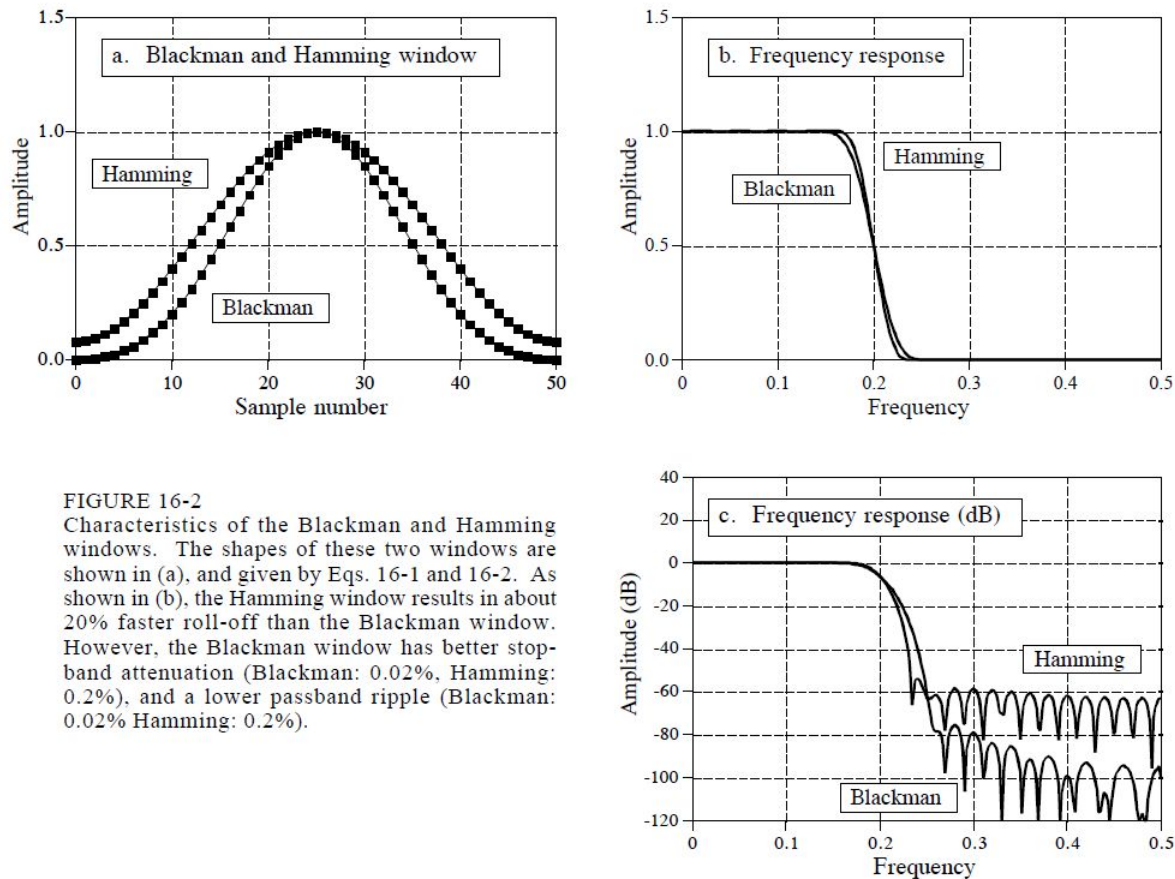


FIGURE 16-2
Characteristics of the Blackman and Hamming windows. The shapes of these two windows are shown in (a), and given by Eqs. 16-1 and 16-2. As shown in (b), the Hamming window results in about 20% faster roll-off than the Blackman window. However, the Blackman window has better stop-band attenuation (Blackman: 0.02%, Hamming: 0.2%), and a lower passband ripple (Blackman: 0.02% Hamming: 0.2%).

Strategy of Windowed-Sinc (contd.)

Other windows

- The **Bartlett window** is a triangle, using straight lines for the taper.
- The **Hanning window**, also called the **raised cosine window**, is given by:

$$w[i] = 0.5 - 0.5 \cos(2\pi i/M)$$

These two windows have about the same roll-off speed as the Hamming, but worse stopband attenuation (Bartlett: ~25dB or 5.6%, Hanning: ~44dB or 0.63%).

Designing the Filter

To design a windowed-sinc, two parameters must be selected: the cutoff frequency, f_c , and the length of the filter kernel, M .

- The cutoff frequency, f_c , is expressed as a fraction of the sampling rate, and therefore must be between 0 and 0.5
- The value of M sets the *roll-off* according to the approximation:

$$M \approx \frac{4}{BW}$$

Where BW is the width of the transition band, measured from where the curve just barely leaves one, to where it almost reaches zero (say 99% to 1% of the curve).

The transition bandwidth is also expressed as a fraction of the sampling frequency, and must be between 0 and 0.5.

This is only an approximation since the roll-off depends on the particular window being used.

Designing the Filter (contd.)

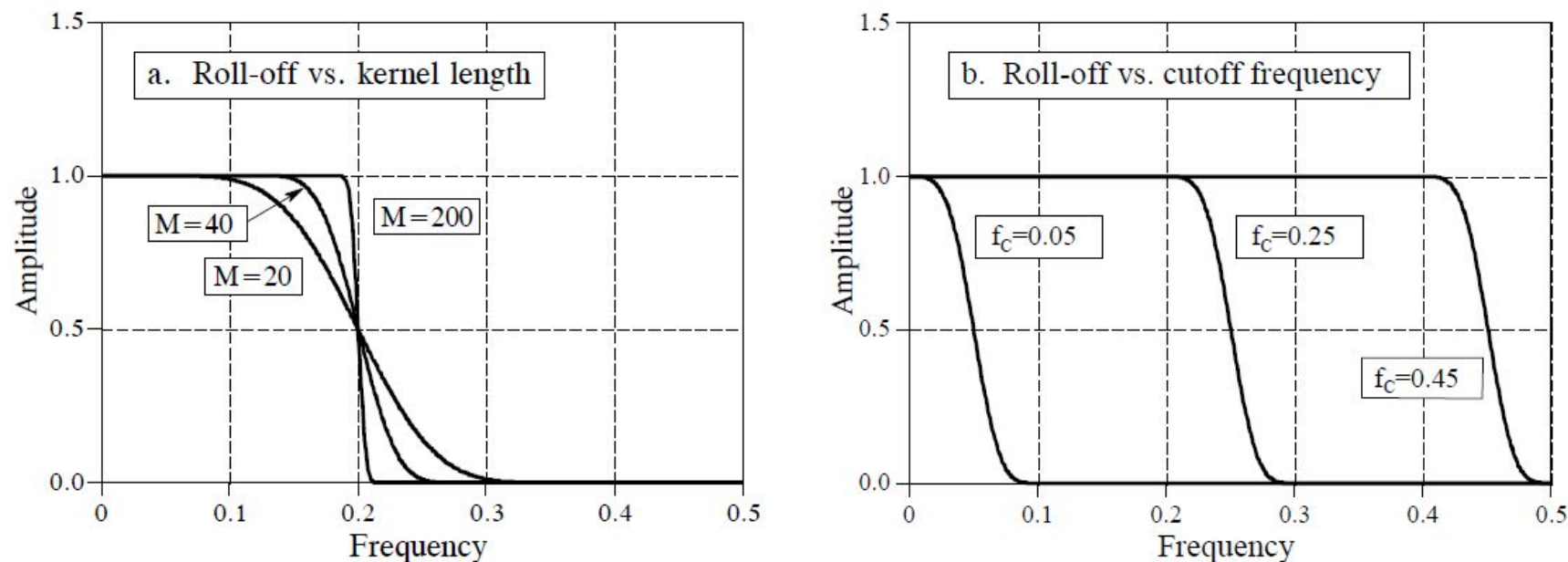
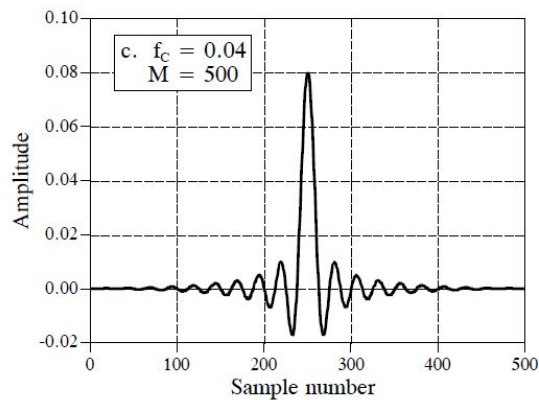
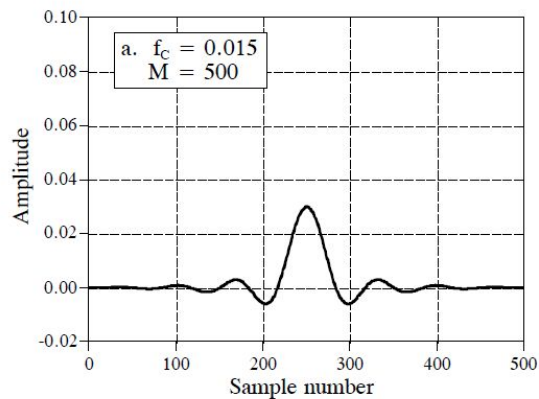


FIGURE 16-3

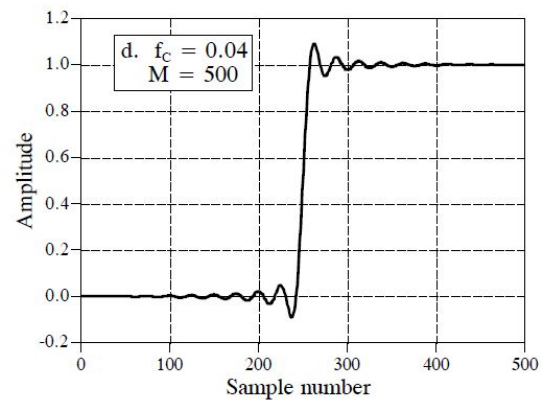
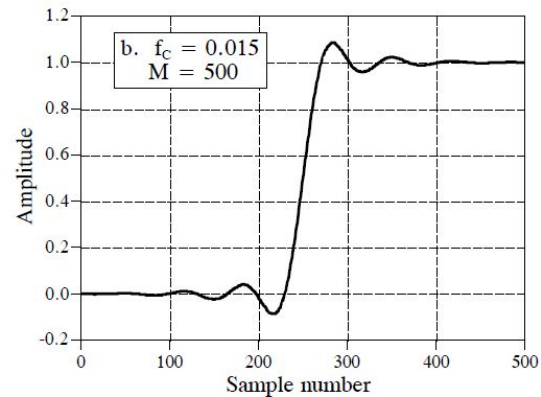
Filter length vs. roll-off of the windowed-sinc filter. As shown in (a), for $M = 20, 40$, and 200 , the transition bandwidths are $BW = 0.2, 0.1$, and 0.02 of the sampling rate, respectively. As shown in (b), the shape of the frequency response does not change with different cutoff frequencies. In (b), $M = 60$.

Designing the Filter (contd.)

Filter kernel



Step response



Designing the Filter (contd.)

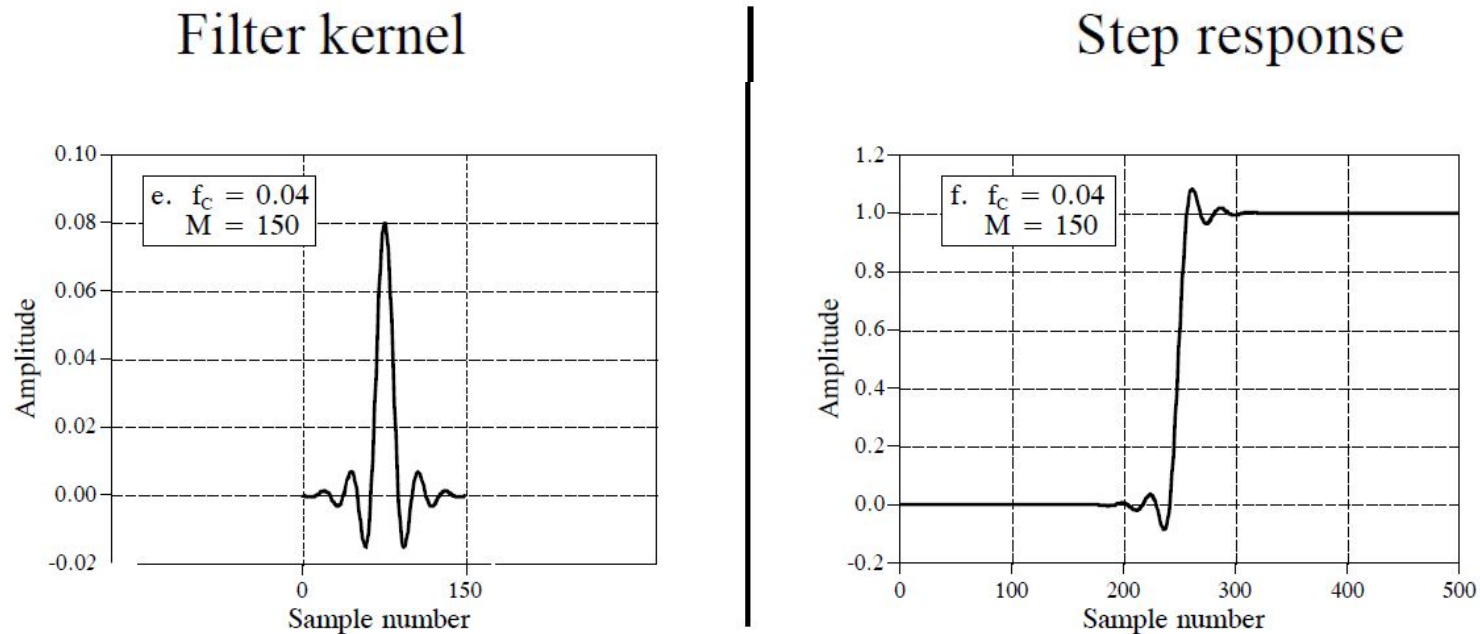


FIGURE 16-4

Example filter kernels and the corresponding step responses. The frequency of the sinusoidal oscillation is approximately equal to the cutoff frequency, f_c , while M determines the kernel length.

Designing the Filter (contd.)

- We know that the time required for a convolution is proportional to the length of the signal.
- Therefore, the aforementioned equation expresses a trade-off between computation time (depends on the value of M) and filter sharpness (depends on the value of BW).
- For instance, the 20% slower roll-off of the Blackman window (as compared with the Hamming) can be compensated for by using a filter kernel 20% longer.
- In other words, it could be said that the **Blackman window is 20% slower to execute than an equivalent roll-off Hamming window.**
- This is important because the execution speed of windowed-sinc filters is already terribly slow.

Designing the Filter (contd.)

- The cutoff frequency of the windowed-sinc filter is measured at the one-half amplitude point.

Why use 0.5 instead of the standard 0.707 (-3dB) used in analog electronics and other digital filters?

Because the windowed-sinc's frequency response is symmetrical between the passband and the stopband. This symmetry makes the windowed-sinc ideal for *spectral inversion*.

Designing the Filter (contd.)

After f_c and M have been selected, the filter kernel is calculated from the relation:

$$h[i] = K \frac{\sin(2\pi f_c(i - M/2))}{i - M/2} \left[0.42 - 0.5 \cos\left(\frac{2\pi i}{M}\right) + 0.08 \cos\left(\frac{4\pi i}{M}\right) \right]$$

Based on the previous discussion, there are three components:

1. The sinc function,
2. The $M/2$ shift,
3. The Blackman window.

For the filter to have unity gain at DC, the constant K must be chosen such that the sum of all the samples is equal to one. In practice, ignore K during the calculation of the filter kernel, and then normalize all of the samples as needed. (Simply divide every sample by the sum of all sample)

Examples of Windowed-Sinc Filters

Electroencephalogram (EEG)

- A measurement of the electrical activity of the brain.
- It can be detected as millivolt level signals appearing on electrodes attached to the surface of the head.
- Each nerve cell in the brain generates small electrical pulses
- Different frequencies in the EEG can be identified with specific mental states.
 - If you close your eyes and relax, the predominant EEG pattern will be a slow oscillation between about 7 and 12 hertz – **Alpha rhythm** – associated with contentment and a decreased level of attention
 - Opening your eyes and looking around causes the EEG to change to the **beta rhythm**, occurring between about 17 and 20 hertz.
- Other frequencies and waveforms are seen in children, different depths of sleep, and various brain disorders such as epilepsy.

Examples of Windowed-Sinc Filters (contd.)

Electroencephalogram (EEG): Separating the alpha from the beta rhythms

Let's assume that the EEG signal has been amplified by analog electronics, and then digitized at a sampling rate of 100 samples per second. We will design a digital low-pass filter with:

- A cutoff frequency of 14 hertz, or 0.14 of the sampling rate
- The transition bandwidth will be set to 4 hertz, or 0.04 of the sampling rate

Examples of Windowed-Sinc Filters (contd.)

Electroencephalogram (EEG): Separating the alpha from the beta rhythms (contd.)

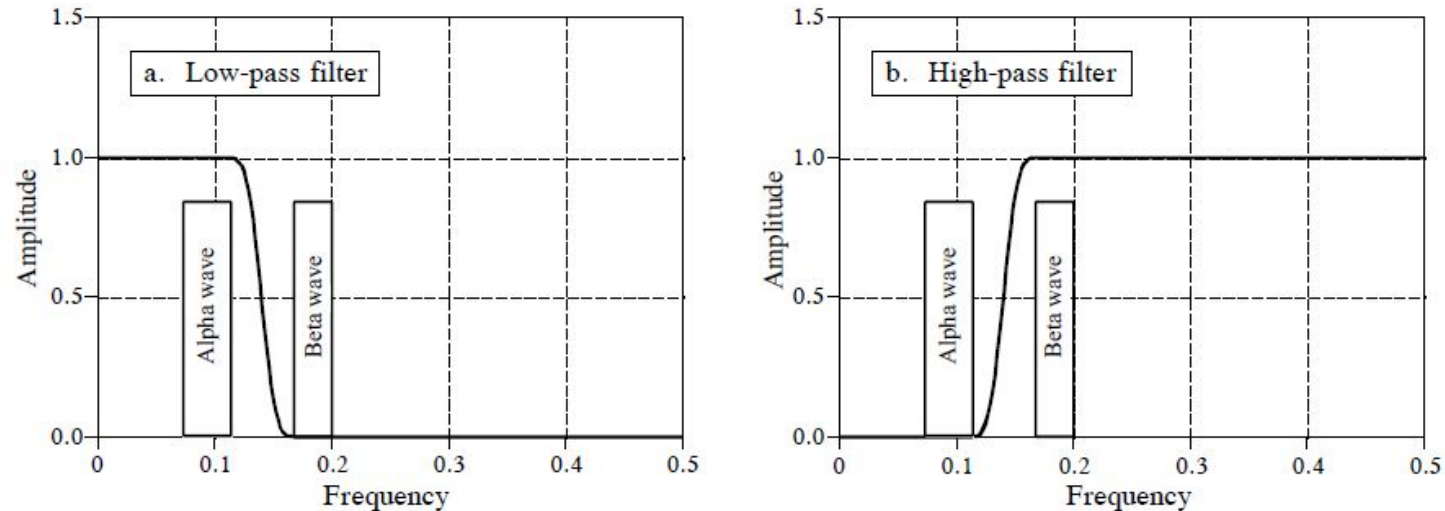


FIGURE 16-5

Example of windowed-sinc filters. The alpha and beta rhythms in an EEG are separated by low-pass and high-pass filters with $M = 100$. The program to implement the low-pass filter is shown in Table 16-1. The program for the high-pass filter is identical, except for a *spectral inversion* of the low-pass filter kernel.

Examples of Windowed-Sinc Filters (contd.)

```
100 'LOW-PASS WINDOWED-SINC FILTER
110 'This program filters 5000 samples with a 101 point windowed-sinc filter,
120 'resulting in 4900 samples of filtered data.
130 '
140 DIM X[4999]                'X[ ] holds the input signal
150 DIM Y[4999]                'Y[ ] holds the output signal
160 DIM H[100]                 'H[ ] holds the filter kernel
170 '
180 PI = 3.14159265
190 FC = .14                    'Set the cutoff frequency (between 0 and 0.5)
200 M% = 100                    'Set filter length (101 points)
210 '
220 GOSUB XXXX                  'Mythical subroutine to load X[ ]
230 '
240 '                            'Calculate the low-pass filter kernel via Eq. 16-4
250 FOR I% = 0 TO 100
260   IF (I%-M%/2) = 0 THEN H[I%] = 2*PI*FC
270   IF (I%-M%/2) <> 0 THEN H[I%] = SIN(2*PI*FC * (I%-M%/2)) / (I%-M%/2)
280   H[I%] = H[I%] * (0.54 - 0.46*COS(2*PI*I%/M%))
290 NEXT I%
300 '
310 SUM = 0                     'Normalize the low-pass filter kernel for
320 FOR I% = 0 TO 100           'unity gain at DC
330   SUM = SUM + H[I%]
340 NEXT I%
350 '
360 FOR I% = 0 TO 100
370   H[I%] = H[I%] / SUM
380 NEXT I%
390 '
400 FOR J% = 100 TO 4999        'Convolve the input signal & filter kernel
410   Y[J%] = 0
420   FOR I% = 0 TO 100
430     Y[J%] = Y[J%] + X[J%-I%] * H[I%]
440   NEXT I%
450 NEXT J%
460 '
470 END
```

Examples of Windowed-Sinc Filters (contd.)

Isolating a signaling tone in an audio signal (such as when a button on a telephone is pressed)

- Let's assume that the signal has been digitized at 10 kHz.
- Our goal is to isolate an 80 hertz band of frequencies centered on 2 kHz.
- In terms of the sampling rate, we want to block all frequencies below 0.196 and above 0.204 (corresponding 1960 hertz and 2040 hertz, respectively)

We need to design a *band-pass filter*. This involve several steps:

1. Two low-pass filters are designed, one with a cutoff at 0.196, and the other with a cutoff at 0.204.
2. The second filter is then spectrally inverted, making it a high-pass filter
3. Next, the two filter kernels are added, resulting in a band-reject filter.
4. Finally, another spectral inversion makes this into the desired band-pass filter.

Examples of Windowed-Sinc Filters (contd.)

Isolating a signaling tone in an audio signal (contd.)

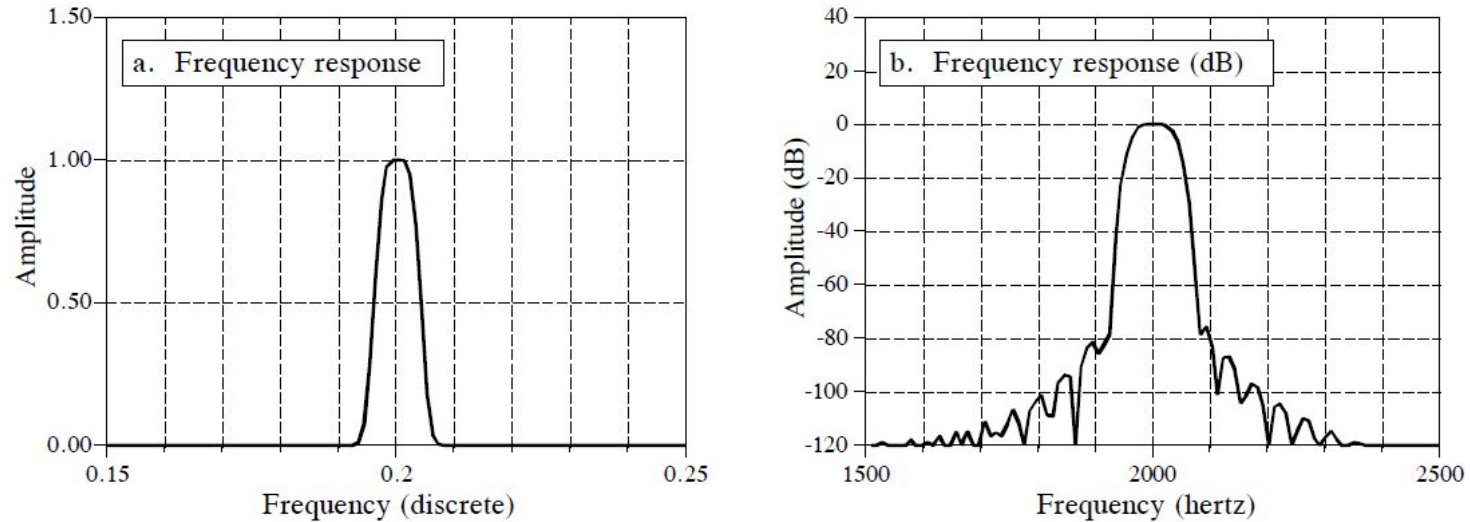


FIGURE 16-6

Example of a windowed-sinc band-pass filter. This filter was designed for a sampling rate of 10 kHz. When referenced to the analog signal, the center frequency of the passband is at 2 kHz, the passband is 80 hertz, and the transition bands are 50 hertz. The windowed-sinc uses 801 points in the filter kernel to achieve this roll-off, and a Blackman window for good stopband attenuation. Figure (a) shows the resulting frequency response on a linear scale, while (b) shows it in decibels. The frequency axis in (a) is expressed as a fraction of the sampling frequency, while (b) is expressed in terms of the analog signal before digitization.

Examples of Windowed-Sinc Filters (contd.)

```
100 'BAND-PASS WINDOWED-SINC FILTER
110 'This program calculates an 801 point band-pass filter kernel
120 '
130 DIM A[800]                'A[ ] workspace for the lower cutoff
140 DIM B[800]                'B[ ] workspace for the upper cutoff
150 DIM H[800]                'H[ ] holds the final filter kernel
160 '
170 PI = 3.1415926
180 M% = 800                  'Set filter kernel length (801 points)
190 '
200 '                          'Calculate the first low-pass filter kernel via Eq. 16-4,
210 FC = 0.196                'with a cutoff frequency of 0.196, store in A[ ]
220 FOR I% = 0 TO 800
230   IF (I%-M%/2) = 0 THEN A[I%] = 2*PI*FC
240   IF (I%-M%/2) <> 0 THEN A[I%] = SIN(2*PI*FC * (I%-M%/2)) / (I%-M%/2)
250   A[I%] = A[I%] * (0.42 - 0.5*COS(2*PI*I%/M%) + 0.08*COS(4*PI*I%/M%))
260 NEXT I%
270 '
280 SUM = 0                   'Normalize the first low-pass filter kernel for
290 FOR I% = 0 TO 800         'unity gain at DC
300   SUM = SUM + A[I%]
310 NEXT I%
320 '
330 FOR I% = 0 TO 800
340   A[I%] = A[I%] / SUM
350 NEXT I%
```


Examples of Windowed-Sinc Filters (contd.)

```
350 NEXT I%
360 '
370 FC = 0.204
380 FOR I% = 0 TO 800
390 IF (I%-M%/2) = 0 THEN B[I%] = 2*PI*FC
400 IF (I%-M%/2) <> 0 THEN B[I%] = SIN(2*PI*FC * (I%-M%/2)) / (I%-M%/2)
410 B[I%] = B[I%] * (0.42 - 0.5*COS(2*PI*I%/M%) + 0.08*COS(4*PI*I%/M%))
420 NEXT I%
430 '
440 SUM = 0
450 FOR I% = 0 TO 800
460 SUM = SUM + B[I%]
470 NEXT I%
480 '
490 FOR I% = 0 TO 800
500 B[I%] = B[I%] / SUM
510 NEXT I%
520 '
530 FOR I% = 0 TO 800
540 B[I%] = - B[I%]
550 NEXT I%
560 B[400] = B[400] + 1
570 '
580 '
590 FOR I% = 0 TO 800
600 H[I%] = A[I%] + B[I%]
610 NEXT I%
620 '
630 FOR I% = 0 TO 800
640 H[I%] = -H[I%]
650 NEXT I%
660 H[400] = H[400] + 1
670 '
680 END
```

'Calculate the second low-pass filter kernel via Eq. 16-4,
'with a cutoff frequency of 0.204, store in B[]

'Normalize the second low-pass filter kernel for
'unity gain at DC

'Change the low-pass filter kernel in B[] into a high-pass
'filter kernel using spectral inversion (as in Fig. 14-5)

'Add the low-pass filter kernel in A[], to the high-pass
'filter kernel in B[], to form a band-reject filter kernel
'stored in H[] (as in Fig. 14-8)

'Change the band-reject filter kernel into a band-pass
'filter kernel by using spectral inversion

'The band-pass filter kernel now resides in H[]